

Tema 13

Introducción a la programación concurrente



Índice

- Introducción
- Tareas en C#
- Programación asíncrona: `async` / `await`
- Programación multihilo
- Sincronización y control de concurrencia
- Patrón Productor-Consumidor
- Buenas prácticas en programación asíncrona y multihilo



- La **conurrencia** es un concepto clave en programación que permite la ejecución simultánea de múltiples operaciones o tareas.
- Gracias a esta técnica, se puede optimizar el rendimiento de los sistemas, haciendo un uso eficiente de los recursos del hardware.

1. Introducción



- En el contexto de aplicaciones modernas, la concurrencia facilita la ejecución de tareas de fondo (como acceso a bases de datos, redes o procesamiento de archivos) sin bloquear otras partes como la interfaz de usuario.

1. Introducción

Características Principales de la Concurrencia

- **Paralelismo:** Consiste en ejecutar múltiples tareas de forma simultánea en diferentes núcleos de la CPU.
- No todas las operaciones concurrentes son paralelas, pero el paralelismo es una de las implementaciones más efectivas de la concurrencia en sistemas con múltiples núcleos.

1. Introducción

- **Asincronía:** Permite la ejecución de tareas sin bloquear el hilo principal.
- Una operación asíncrona puede liberarse del hilo principal para continuar en segundo plano y notificar su resultado cuando esté disponible.

1. Introducción

- **Indeterminismo:** A diferencia de la programación secuencial, en la programación concurrente **no sabemos el orden** que va a seguir el flujo de la ejecución.
- Esto implica que ante unas mismas entradas, sucesivas ejecuciones pueden dar lugar a resultados completamente diferentes.

1. Introducción

- **Sincronización:** Debido a este indeterminismo, será imprescindible utilizar mecanismos para sincronizarnos durante la ejecución.
- Debemos sincronizar correctamente el acceso a recursos compartidos (por ejemplo, las variables), las esperas entre diferentes procesos o el paso de mensajes.

1. Introducción

Problemas Comunes en la Programación Concurrente

- La concurrencia introduce complejidades en el desarrollo de software, especialmente cuando diferentes tareas o hilos intentan acceder y modificar los mismos recursos.
- Vamos a ver una lista de los principales problemas que deberemos afrontar.

1. Introducción

- **Condiciones de Carrera (Race Conditions):** Ocurren cuando múltiples hilos o tareas acceden y manipulan un recurso compartido de forma simultánea.
- El orden en que los hilos acceden al recurso afecta el resultado final, lo que genera inconsistencias y errores impredecibles.
- Por ejemplo, si 2 hilos diferentes modifican la misma variable, no sabemos qué valor almacenará al final.

1. Introducción

- **Deadlock:** Sucede cuando dos o más hilos quedan esperando indefinidamente a que otro hilo libere un recurso, bloqueando su progreso y creando un ciclo sin resolución. Los deadlocks ocurren generalmente cuando los hilos necesitan múltiples recursos que son compartidos.
- **Livelock:** Similar al deadlock, pero en lugar de estar bloqueados, los hilos están ocupados respondiendo a las acciones de otros hilos y nunca logran completar su tarea.

1. Introducción

- **Inanición (Starvation):** En sistemas donde múltiples hilos compiten por los mismos recursos, algunos pueden quedarse sin acceso suficiente para completar sus tareas.
- Esto es especialmente crítico si hilos de mayor prioridad siempre obtienen los recursos primero, provocando que los de menor prioridad nunca puedan completar sus tareas.

1. Introducción

- Los programas concurrentes son **difíciles de escribir**. Debemos controlar el orden, acceso a recursos, sincronización...
- Además, su depuración es mucho más compleja que la secuencial debido a las múltiples bifurcaciones de caminos que puede tomar el flujo de ejecución.
- C# proporciona varias herramientas y características para implementar concurrencia de forma segura y efectiva que vamos a conocer en este tema.

1. Introducción

- Las **tareas** en C# (Task) son una forma moderna y versátil de manejar la programación concurrente y asíncrona.
- Una tarea representa una operación que se puede ejecutar en paralelo o en segundo plano, liberando el hilo principal de la aplicación para otras operaciones.
- En vez de gestionar directamente los hilos, las tareas permiten trabajar con abstracciones más sencillas y controladas, automatizando la asignación de recursos de procesamiento.

2. Tareas en C#

Creación y Ejecución de Tareas

- Las tareas en C# se crean utilizando la clase Task del espacio de nombres `System.Threading.Tasks`.
- Para ejecutar una tarea, debe tener asociado un delegado de tipo `Action`.

2. Tareas en C#

- Podemos crear las tareas mediante el constructor `new Task(Action)`.
- Para lanzar una tarea, utilizamos el método `Start()`.
- También podemos crear y ejecutar directamente una tarea mediante el uso de `Task.Run`:

2. Tareas en C#

```
1 Action actionDelegate = () => Console.WriteLine("Delegac  
2 Task task = new Task(actionDelegate); // Crea la tarea,
```

```
1 Action actionDelegate = () => Console.WriteLine("Delega
```

- Como hemos dicho en la introducción, la programación concurrente provoca resultados impredecibles y cambiantes.
- Vamos a ver un ejemplo en el que creamos 100 tareas que imprimen por pantalla su número de ejecución:

2. Tareas en C#

```
1 static void Main(string[] args)
2 {
3     for (int i = 0; i < 100; i++)
4     {
5         Task t = Task.Run(() => Console.WriteLine("tar
6     })
7
8     Console.WriteLine("fin");
9 }
```

- En la programación secuencial, cabría esperar que cada una de las tareas imprimiese su número y además estos números fuesen aumentando de 1 en 1.
- Sin embargo, cuando ejecutamos este bucle, obtenemos cada vez un resultado diferente.
- Ni siquiera el número de tareas que terminan a tiempo (antes de que finalice el programa) es constante o predecible.

2. Tareas en C#

- Cada ejecución devuelve un resultado diferente y aparentemente poco coherente:

2. Tareas en C#



- Es por ello que en ocasiones es imprescindible utilizar mecanismos de sincronización para poder gestionar el flujo.
- Para controlar tareas, C# proporciona métodos que permiten esperar a que una o varias tareas terminen antes de continuar con la ejecución del código.

2. Tareas en C#

- **Wait**: Bloquea el hilo actual hasta que la tarea específica termine.
- **WaitAll**: Espera a que un conjunto de tareas termine.

2. Tareas en C#

```
1 Task myTask = Task.Run(() => DoWork());
2 myTask.Wait(); // Espera a que la tarea termine
```

```
1 Task[] tasks = new Task[3];
2 for (int i = 0; i < 3; i++)
3 {
4     tasks[i] = Task.Run(() => DoWork());
5 }
```

- **WhenAll**: Devuelve una tarea que se completa cuando todas las tareas pasadas como argumento han finalizado.
- **ContinueWith**: Las continuaciones permiten ejecutar una tarea o acción después de que una tarea previa finalice.

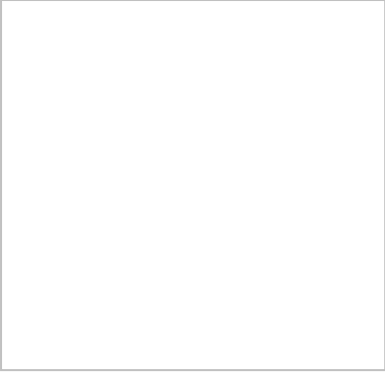
2. Tareas en C#

```
1 Task endTask = Task.WhenAll  
  
1 Task initialTask = Task.Run(() => {  
2     Console.WriteLine("Tarea inicial");  
3 });  
4 Task continuation = initialTask.ContinueWith((prev  
5     Console.WriteLine("Tarea de continuación");  
6 ));
```

- Con estos mecanismos, podemos replantear el ejemplo anterior.
- Si utilizamos `WaitAll`, nos aseguramos que todas las tareas se ejecuten antes de finalizar el programa, aunque no tenemos control de su orden ni del estado de la variable cuando se ejecuten.

2. Tareas en C#

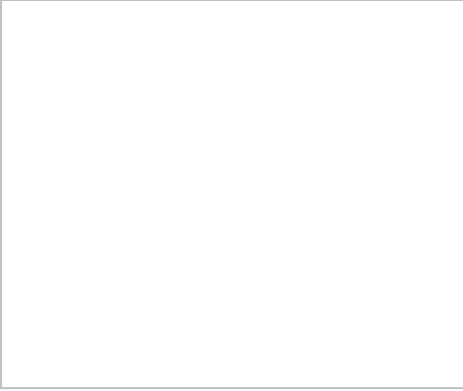
```
1 var tasks = new List<Task>()
2
3 for (int i = 0; i < 100; i++)
4 {
5     tasks.Add(Task.Run(() => {
6         // ...
7     }));
8 Task.WaitAll(tasks);
```



- Si utilizamos un `Wait` en cada una de las tareas, sí que tendremos la certeza de que cada una se ejecuta completamente antes de continuar con la siguiente.
- De esta manera también tenemos control sobre el estado de la variable `i` cuando la utilizan.

2. Tareas en C#

```
1 var tasks = new List<Task>()
2
3 for (int i = 0; i < 100; i++)
4 {
5     Task t = Task.Run(() =>
6         tasks.Add(t);
7     t.Wait();
8 }
```



Manejo de Excepciones en Tareas

- Las tareas que arrojan excepciones deben ser controladas para evitar errores inesperados.
- En C#, las excepciones lanzadas dentro de una tarea se capturan automáticamente y se pueden manejar utilizando try-catch alrededor de `task.Wait()` o con la propiedad `task.Exception`.

2. Tareas en C#

Ejemplo

2. Tareas en C#

```
1  try {  
2      Task faultyTask = Task.Run(() => {  
3          throw new InvalidOperationException("Error  
4      });  
5      faultyTask.Wait();  
6  } catch (AggregateException ex) {  
7      foreach (var innerException in ex.InnerExceptio  
8          Console.WriteLine(innerException.Message);  
9  }
```

Resultados de Tareas: Task<TResult>

- Además de tareas que únicamente realizan acciones sin devolver valores, también podemos crear tareas que retornan un valor.
- Para las tareas que deben devolver un valor, se usa la clase genérica Task<TResult> y ejecutan un Func en vez de un Action.
- Esto permite que una tarea asíncrona retorne un resultado específico una vez que termina.

2. Tareas en C#

Ejemplo

- En el siguiente punto veremos como recoger en una variable el resultado de la ejecución de una tarea.

2. Tareas en C#

```
1 Task<double> = Task.Run(()=>{  
2     Random r = new Random(DateTime.Now.Mil  
3     return r.NextDouble() * 100;  
.. }
```

Vídeo práctico

2. Tareas en C#

Vídeo práctico - código

2. Tareas en C#

```
1 Task task = new Task(() => Console.WriteLine("tarea co
2 task.Start();
3
4 Console.WriteLine("hilo principal");
5
6 Task[] tasks = new Task[10];
7
8 for (int i = 0; i < 10; i++)
9 {
10     tasks[i] = Task.Run(() => Console.WriteLine("El cu
11 }
12
13 Task.WaitAll(tasks);
14
15 Task<int> intTask = Task.Run(() =>
16 {
17     return DateTime.Now.Millisecond;
18 });
19
20 try
```

- La **programación asíncrona** nos permite realizar tareas largas en segundo plano mientras continúa la ejecución del hilo principal.
- El uso de `async` y `await` en C# es fundamental para desarrollar aplicaciones eficientes que realizan tareas en segundo plano, como puede ser el caso de aplicaciones con interfaz de usuario o servidores web.

3. Programación asíncrona: `async` / `await`

Concepto de `async` y `await`

- **`async`**: El modificador `async` se utiliza en la firma del método para indicar que el método contiene operaciones asíncronas.
- El método `async` generalmente retorna un tipo de `Task` o `Task<T>`. Esto indica que el método es asíncrono y puede devolver un valor (en el caso de `Task<T>`) o no (con `Task`).

3. Programación asíncrona: `async` / `await`

- Sin embargo, no hace que el método sea asíncrono por sí mismo: esto se logra mediante el uso de la palabra clave `await` en su interior.

`await`: `await` se utiliza dentro de un método marcado como `async` para pausar su ejecución hasta que una operación asíncrona termine, sin bloquear el hilo actual.

3. Programación asíncrona: **`async` / `await`**

Sintaxis Básica de `async` y `await`

- Para crear un método asíncrono, este debe ser de tipo `void` o `Task` y se le debe añadir el modificador `async` en la declaración del método.
- Para ejecutar operaciones de manera asíncrona, estas deben ir precedidas de `await`.

3. Programación asíncrona: `async` / `await`

```
1 public async Task PerformCalculationAsync()  
2 {  
3     Console.WriteLine("Cálculo iniciado");  
4     await Task.Delay(2000); // Simulación de una t  
5     Console.WriteLine("Cálculo finalizado");  
6 }
```

- En el ejemplo anterior, `Task.Delay(2000)` simula una operación que tarda 2 segundos, durante los cuales la ejecución se pausa en este método **sin bloquear el hilo que lo llamó**.

3. Programación asíncrona: async / await

Ejemplo "real"

- En una operación real, podríamos querer, por ejemplo, leer datos de un archivo o realizar una solicitud a una API de manera asíncrona.
- Al llamar a `GetStringAsync`, `await` suspende la ejecución hasta que se obtiene la respuesta de la API sin bloquear el hilo, lo que es especialmente útil en aplicaciones con interfaz de usuario.

3. Programación asíncrona: `async` / `await`

```
1 public static async Task<string> FetchDataFromAPIAs  
2 {  
3     HttpClient client = new HttpClient();
```

4

```
string result = await client.GetStringAsync("ht
```

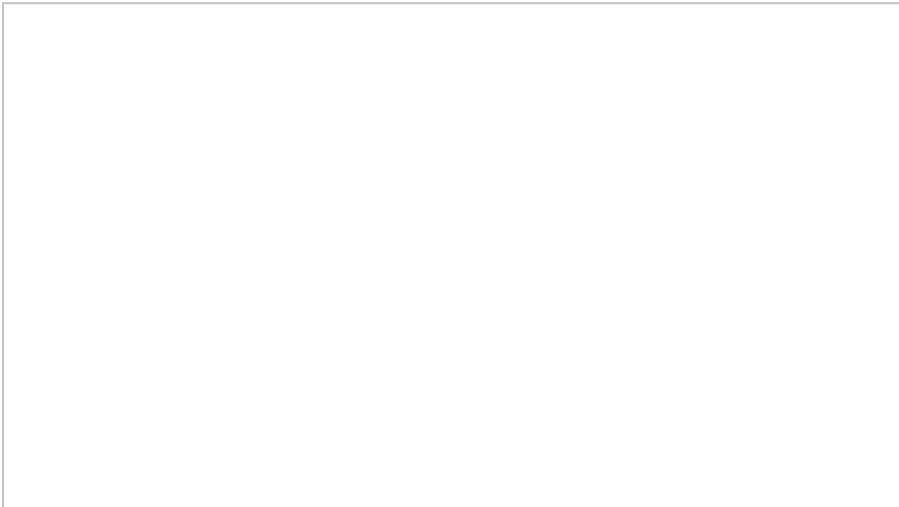
- Para ver el efecto de la asincronía en la tarea, vamos a llamarla desde nuestro Main.
- Mientras la tarea esté realizando la llamada a la API de prueba que hemos elegido, el Main va a imprimir * por consola:

3. Programación asíncrona: async / await

```
1  static void Main(string[] args)
2  {
3      var task = FetchDataFromAPIAsy
4
5      while (!task.IsCompleted)
6      {
7          Console.Write("*");
8      }
9
10     Console.WriteLine(task.Result
```

- Cuando ejecutemos, el resultado será la impresión de numerosos * seguidos del resultado de la llamada a la API.
- Si la tarea no fuese asíncrona, bloquearía el Main y no sería capaz de pintar los *.

3. Programación asíncrona: async / await



Importancia de `async` y `await` en Aplicaciones de Escritorio y Web

- En aplicaciones de escritorio o móviles, `async` y `await` son esenciales para mantener la interfaz de usuario receptiva mientras se ejecutan operaciones largas en segundo plano.
- Sin ellos, la interfaz se congelaría cada vez que se realizan tareas con carga alta de procesamiento, proporcionando una experiencia de usuario deficiente.

3. Programación asíncrona: `async` / `await`

- En aplicaciones de servidor, como ASP.NET, la programación asíncrona permite que los hilos del servidor estén disponibles para otros usuarios mientras el sistema espera una respuesta, aumentando la capacidad de respuesta de la aplicación y su escalabilidad.

3. Programación asíncrona: async / await

Vídeo práctico

3. Programación asíncrona: `async` / `await`

Vídeo práctico - código

3. Programación asíncrona: async / await

```
1  public static void Main()  
2  {  
3      var loadTask = Task.Run(LoadDataAsync);  
4  
5      while (!loadTask.IsCompleted)  
6      {  
7          Console.Write("*");  
8      }  
9  
10     Console.WriteLine(loadTask.Result);  
11 }  
12  
13 public static async Task<string> LoadDataAsync(  
14 {  
15     Console.WriteLine("Inicio de la descarga");  
16     await Task.Delay(1000);  
17     return "Descarga finalizada";  
18 }
```

- La **programación multihilo** en C# permite que un programa ejecute múltiples tareas de manera simultánea usando varios hilos.
- Esta práctica mejora el rendimiento y la eficiencia en situaciones en las que se necesitan realizar varias operaciones en paralelo.

4. Programación multihilo

- Esta capacidad resulta especialmente útil en aplicaciones que manejan procesos intensivos, como operaciones de cálculo complejas, procesamiento de archivos o interacción con bases de datos, donde es posible que el programa esté esperando recursos externos o procesos.

4. Programación multihilo

Concepto de Hilos

- Un **hilo** es la unidad básica de procesamiento que el sistema operativo puede manejar de manera independiente.
- Un **programa** C# puede tener varios hilos en ejecución al mismo tiempo, y cada uno puede manejar diferentes tareas de la aplicación.
- La clase Thread en C# proporciona métodos y propiedades para crear y gestionar estos hilos.

4. Programación multihilo

Creación de Hilos en C#

- La creación de hilos en C# se realiza instanciando un objeto Thread y pasando un método delegado que contiene el código que el hilo ejecutará.
- Para que el hilo comience, se llama al método Start ().

4. Programación multihilo

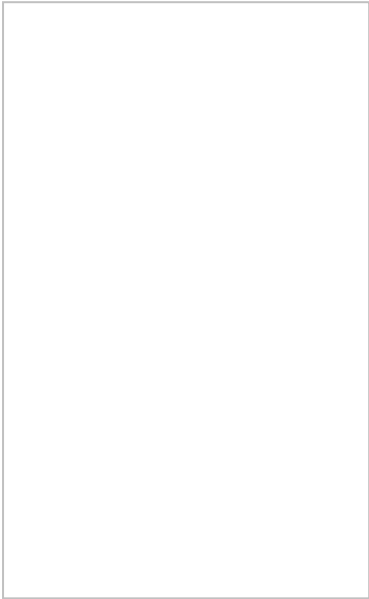
Ejemplo

4. Programación multihilo

```
1  public static void SimulateWork(int id, int dela
2  {
3      for (int i = 0; i < 10; i++)
4      {
5          Console.WriteLine("Hilo: {0}. Tarea: {1}
6          Thread.Sleep(delay); // Simula trabajo c
7      }
8  }
9
10 static void Main(string[] args)
11 {
12     Thread thread1 = new Thread(() => SimulateWo
13     Thread thread2 = new Thread(() => SimulateWo
14
15     thread1.Start();
16     thread2.Start();
17 }
```

- En el ejemplo anterior, creamos 2 hilos que ejecutan el mismo método (que simula una carga de trabajo y escribe 10 veces por pantalla).
- Los hilos se ejecutan de manera simultánea y paralela:

4. Programación multihilo



- Los hilos nos ofrecen mayor control que las tareas, pudiendo asignar prioridad, agruparlos en *pools*, realizar uniones avanzadas, modificar su estado a voluntad...
- Sin embargo, también suponen una mayor dificultad ya que trabajan a un nivel más bajo que las Task, que ofrecen una abstracción mayor dejando que realice por nosotros algunas tareas de su gestión.

4. Programación multihilo

- Un reto en la programación multihilo es la **sincronización de hilos**.
- Esta hace referencia a controlar el acceso concurrente a los recursos compartidos para evitar problemas como **condiciones de carrera y interbloqueos**.

5. Sincronización y control de concurrencia

- Una condición de carrera ocurre cuando varios hilos intentan acceder y modificar un recurso compartido al mismo tiempo, lo que produce resultados inesperados.
- C# ofrece herramientas y estructuras de sincronización como lock, Monitor, Mutex, Semaphore o SemaphoreSlim para asegurar que los hilos accedan a los recursos de manera controlada.

5. Sincronización y control de concurrencia

lock

- El lock es el mecanismo de sincronización más sencillo.
- Permite que solo un hilo entre en una sección crítica del código, protegiendo los recursos compartidos.
- Este mecanismo es ideal cuando varios hilos modifican el valor de una variable, por ejemplo.

5. Sincronización y control de concurrencia

Ejemplo:

- En este ejemplo, únicamente 1 hilo va a poder ejecutar el código contenido en el bloque `lock`. Hasta que no termine, el resto de hilos que quieren acceder quedan en espera.

5. Sincronización y control de concurrencia

```
1 private object lockObj = new object();
2
3 public void AccessResource()
4 {
5     lock (lockObj)
6     {
7         Console.WriteLine("Acceso controlado");
8     }
9 }
```

```
8      + Thread.CurrentThread.ManagedThr  
9  
10     // Operaciones en la sección crítica  
11
```

Monitor

- Monitor es similar a lock, pero ofrece más control sobre el bloqueo.
- Se puede usar para adquirir y liberar explícitamente el bloqueo, proporcionando funcionalidades avanzadas como `Monitor.Wait` y `Monitor.Pulse` para coordinar la interacción entre hilos.

5. Sincronización y control de concurrencia

Ejemplo:

- El resultado es similar al lock, pero tenemos más opciones de control utilizando Monitor.

5. Sincronización y control de concurrencia

```
1 private object monitorObj = new object();
2
3 public void AccessWithMonitor()
4 {
5     Monitor.Enter(monitorObj);
6     try
7     {
```

```
8      Console.WriteLine("Acceso controlado c  
9      // Código de sección crítica  
10     }  
11     finally  
12     {  
13         Monitor.Exit(monitorObj);  
14     }
```

Mutex

- Mutex es un mecanismo de sincronización de mayor alcance que lock, y puede utilizarse en sincronización entre procesos (no solo entre hilos de un mismo proceso).
- Sin embargo, es más lento que lock y se recomienda para escenarios donde la sincronización se da entre aplicaciones distintas.

5. Sincronización y control de concurrencia

Ejemplo:

- El objeto Mutex se puede compartir entre diferentes procesos.

5. Sincronización y control de concurrencia

```
1  private static Mutex mutex = new Mutex();
2
3  public void AccessWithMutex()
4  {
5      mutex.WaitOne();
6      try
7      {
8          Console.WriteLine("Acceso controlado |
9          // Código de sección crítica
```

```
10 }  
11 finally  
12 {  
13     mutex.ReleaseMutex();  
14 }
```

Semaphore y SemaphoreSlim

- Semaphore y SemaphoreSlim permiten que varios hilos accedan a un recurso compartido hasta un límite especificado.
- Este mecanismo es ideal cuando un recurso puede soportar más de un acceso concurrente.
- SemaphoreSlim es una versión más ligera de Semaphore y se usa solo dentro de la misma aplicación.

5. Sincronización y control de concurrencia

Ejemplo:

- Los semáforos permiten que más de 1 hilo accedan a los recursos compartidos.

5. Sincronización y control de concurrencia

```
1 // Permite que accedan al recurso hasta 3 hilos
2 private static SemaphoreSlim semaphore = new Semaphr
3
4
5 public async Task AccessWithSemaphore()
6 {
```

```
7    await semaphore.WaitAsync();
8    try
9    {
10       Console.WriteLine("Acceso controlado por Se
11       // Código de sección crítica
12    }
13    finally
14    {
15       semaphore.Release();
16    }
```

Vídeo práctico

5. Sincronización y control de concurrencia

Vídeo práctico - código

5. Sincronización y control de concurrencia

```
1  private static object _lock = new object();
2
3  public static void Main()
4  {
5      Thread thread = new Thread(() => PaintVerticalLine(
6      Thread thread2 = new Thread(() => PaintHorizontalLi
7
8      thread.Start();
9      thread2.Start();
10 }
11
12 public static void PaintHorizontalLine(int pos, int del
13 {
14     int i = 0;
15     while (true)
16     {
17         Monitor.Enter(_lock);
18
19         Console.SetCursorPosition(pos, i);
20         Console.Write(" ");
21         i++;
```

- El **Patrón Productor-Consumidor** es un ejemplo clásico en programación concurrente para gestionar el flujo de datos entre componentes que producen y consumen información de forma asíncrona.
- Este patrón utiliza una cola o buffer compartido donde los productores colocan los datos y los consumidores los procesan.
- Este patrón permite que múltiples procesos o hilos trabajen de manera coordinada sin interferencias directas.

6. Patrón productor-consumidor

- El objetivo principal del patrón es permitir la producción y el consumo de datos de forma concurrente sin problemas de sincronización ni pérdida de datos.
- Al desacoplar a los productores de los consumidores, se permite que ambos operen a diferentes velocidades y se optimiza el uso de recursos.

6. Patrón productor-consumidor

Componentes del Patrón Productor-Consumidor

- **Productor:** Genera datos o tareas y las coloca en una cola o buffer compartido.
- **Consumidor:** Toma datos o tareas de la cola para procesarlos.

6. Patrón productor-consumidor

- **Buffer o Cola:** Estructura de datos compartida que permite que los productores y consumidores se comuniquen.
- Este buffer es típicamente una cola (FIFO) que almacena los elementos generados por los productores y que los consumidores extraen en orden.

6. Patrón productor-consumidor

Implementación

- En este ejemplo, el productor generará números aleatorios y los colocará en una cola compartida, mientras que el consumidor extraerá estos números y los procesará.
- Usaremos Mutex para asegurar que solo un hilo acceda a la cola a la vez y SemaphoreSlim para señalar cuándo hay elementos disponibles para consumir.

6. Patrón productor-consumidor

- En primer lugar, definimos los elementos compartidos: el buffer, el Mutex, el semáforo y un contador.

6. Patrón productor-consumidor

```
1 private static Queue<int> buffer = new Queue<int>();  
2 private static readonly Mutex mutex = new Mutex();  
3 private static readonly SemaphoreSlim itemsAvailable = new S
```

- Después escribimos el código del productor.

6. Patrón productor-consumidor

```
1  public static void Producer()  
2  {  
3      for (int i = 0; i < 10; i++)  
4      {  
5          int data = new Random().Next(100);  
6          mutex.WaitOne(); // Bloquea el acceso a la cola  
7  
8          try  
9          {  
10             // Comprueba que no se sobrepase el límite  
11             if (buffer.Count < bufferLimit)  
12             {  
13                 buffer.Enqueue(data);  
14                 Console.WriteLine("Produciendo: " + data);  
15                 itemsAvailable.Release(); // Notifica a  
16             }  
17             else  
18             {  
19                 Console.WriteLine("Buffer lleno. Produc  
20             }
```

- Este productor genera un número aleatorio y bloquea el acceso al Mutex para realizar su código.
- Si hay hueco en el buffer, añade el número y avisa de que hay un nuevo dato disponible. Si no hay hueco, espera a que lo haya.
- Una vez ha realizado su operación, libera el Mutex.

6. Patrón productor-consumidor

- El código del consumidor es el siguiente:

6. Patrón productor-consumidor

```
1  public static void Consumer()  
2  {  
3      for (int i = 0; i < 10; i++)  
4      {  
5          itemsAvailable.Wait(); // Espera a que haya un  
6          mutex.WaitOne(); // Bloquea el acceso a la cola  
7  
8          try  
9          {  
10             if (buffer.Count > 0)  
11             {  
12                 int data = buffer.Dequeue();  
13                 Console.WriteLine("Consumiendo: " + data);  
14             }  
15         }  
16         finally  
17         {  
18             mutex.ReleaseMutex(); // Libera el bloqueo  
19         }  
20  
21         Thread.Sleep(1000); // Simula el tiempo de proc
```

- Este consumidor espera a que haya un dato disponible en primer lugar. Si no hay ningún dato, se queda esperando hasta que lo haya, momento en el que continuará automáticamente.
- Después bloquea el acceso al Mutex para realizar su operativa. Esta operativa simplemente consume un dato y lo imprime.
- Cuando ha terminado, libera el Mutex.

6. Patrón productor-consumidor

- Únicamente nos queda crear las tareas del productor y el consumidor en el Main:

6. Patrón productor-consumidor

```
1 public static void Main()  
2 {  
3     // Crear y ejecutar las tareas de productor  
4     Task producerTask = Task.Run(() => Producer(  
5     Task consumerTask = Task.Run(() => Consumer(  
6  
7     // Esperar a que ambas tareas terminen  
8     Task.WaitAll(producerTask, consumerTask);  
9 }
```

- Cuando ejecutamos, podemos ver como el productor y el consumidor van realizando sus tareas, cada una con su tiempo de espera (simulación de carga de trabajo):

6. Patrón productor-consumidor



Nota: *C# nos ofrece las clases `BlockingCollection` y `ConcurrentQueue` entre otras, que se encargan de realizar la gestión de la concurrencia de manera efectiva, por lo que no tendríamos que hacer uso del `Mutex` y el semáforo.*

Sin embargo, el objetivo de este punto era por una parte conocer el patrón clásico y por otra afianzar los conceptos vistos anteriormente.

6. Patrón productor-consumidor

- La programación concurrente conlleva desafíos relacionados con el control de concurrencia, el rendimiento y la complejidad del código.
- Debido a la dificultad de escribir y depurar código concurrente, se vuelve imprescindible seguir una serie de pautas y buenas prácticas.
- A continuación se describen algunas de las mejores prácticas para escribir código seguro y eficiente en programación concurrente.

7. Buenas prácticas en programación asíncrona y multihilo

USA ASYNC Y AWAIT DONDE SEA POSIBLE

- Los métodos `async` y `await` simplifican el código asíncrono, haciéndolo más legible y evitando bloqueos de hilos.
- Asegúrate de que los métodos `async` devuelvan `Task` (o `Task<T>` para métodos con retorno) para que puedan encadenarse correctamente.

7. Buenas prácticas en programación asíncrona y multihilo

PREFIERE OBJETOS DE SINCRONIZACIÓN DE ALTO NIVEL

- Siempre que sea posible, usa herramientas de sincronización de alto nivel como `SemaphoreSlim`, `BlockingCollection`, o `ConcurrentQueue` en lugar de crear manualmente bloqueos con `lock`.
- Estas clases están optimizadas para la concurrencia y suelen ser más seguras y menos propensas a errores que los bloqueos manuales.

7. Buenas prácticas en programación asíncrona y multihilo

MANEJA EXCEPCIONES EN HILOS ASÍNCRONOS Y TAREAS

- En programación multihilo, las excepciones en los hilos secundarios no se propagan al hilo principal.
- Para capturarlas, usa bloques try-catch dentro de cada tarea o hilo.
- En tareas, puedes capturar excepciones al acceder a la propiedad `Exception` de `Task`, o en `async/await`, maneja las excepciones con try-catch al esperar las tareas.

7. Buenas prácticas en programación asíncrona y multihilo

EVITA EL USO INNECESARIO DE HILOS

- No crees hilos manualmente si puedes usar Task. Esto permite que el sistema administre el uso de hilos de manera eficiente.
- Los hilos manuales consumen más recursos y no se adaptan tan bien a la escalabilidad como las Task.

7. Buenas prácticas en programación asíncrona y multihilo

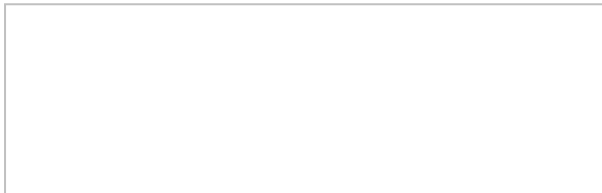
OPTIMIZA EL ACCESO A RECURSOS COMPARTIDOS

- Si tu aplicación requiere acceso a datos compartidos, organiza el código de manera que minimice las modificaciones simultáneas de estos datos.
- Considera el uso de patrones como el patrón Productor-Consumidor, junto con estructuras como ConcurrentQueue, que están optimizadas para escenarios de acceso concurrente.

7. Buenas prácticas en programación asíncrona y multihilo

Diapositiva final

En este tema hemos visto una introducción a la programación asíncrona, la programación concurrente y la sincronización de hilos y tareas. Es un tema complejo, pero que resulta fundamental en la programación actual.



Speaker notes